

RUNNER'S UP PRESENTS
BLU-RESERVE

Experience fine dining at its best

PROBLEM STATEMENT

In large organisational cafeterias such as Riviera, finding available seating during peak hours becomes difficult due to high footfall and lack of structured seat management. This leads to overcrowding, inconvenience for employees, and inefficient utilisation of cafeteria space.

Blu-Reserve is a digital cafeteria seat booking application that allows users to pre-book seats for specific time slots. The system supports time-based reservations, enforces capacity constraints, and provides secure access, enabling better crowd management and an improved user experience.

Restaurant Seat Booking

Experience dining like never before

LOGIN AS IBM EMPLOYEE

LOGIN AS MANAGER

LOGIN AS DEMO USER

Special features

Enables advance seat booking with time-based occupancy management to prevent overcrowding and ensure fair access to cafeteria seating.

The application provides automated notifications through SMS and Email to keep users informed in real time.

The application integrates IBM W3 Single Sign-On for secure authentication.

Supports features such as booking cancellation and modification

Separate dashboard for cafeteria managers for better maintenance

SOLUTION OVERVIEW

1

Blu-Reserve is a smart, digital cafeteria seat booking system designed to transform the traditional cafeteria experience through structured, technology-driven management of seating and occupancy.

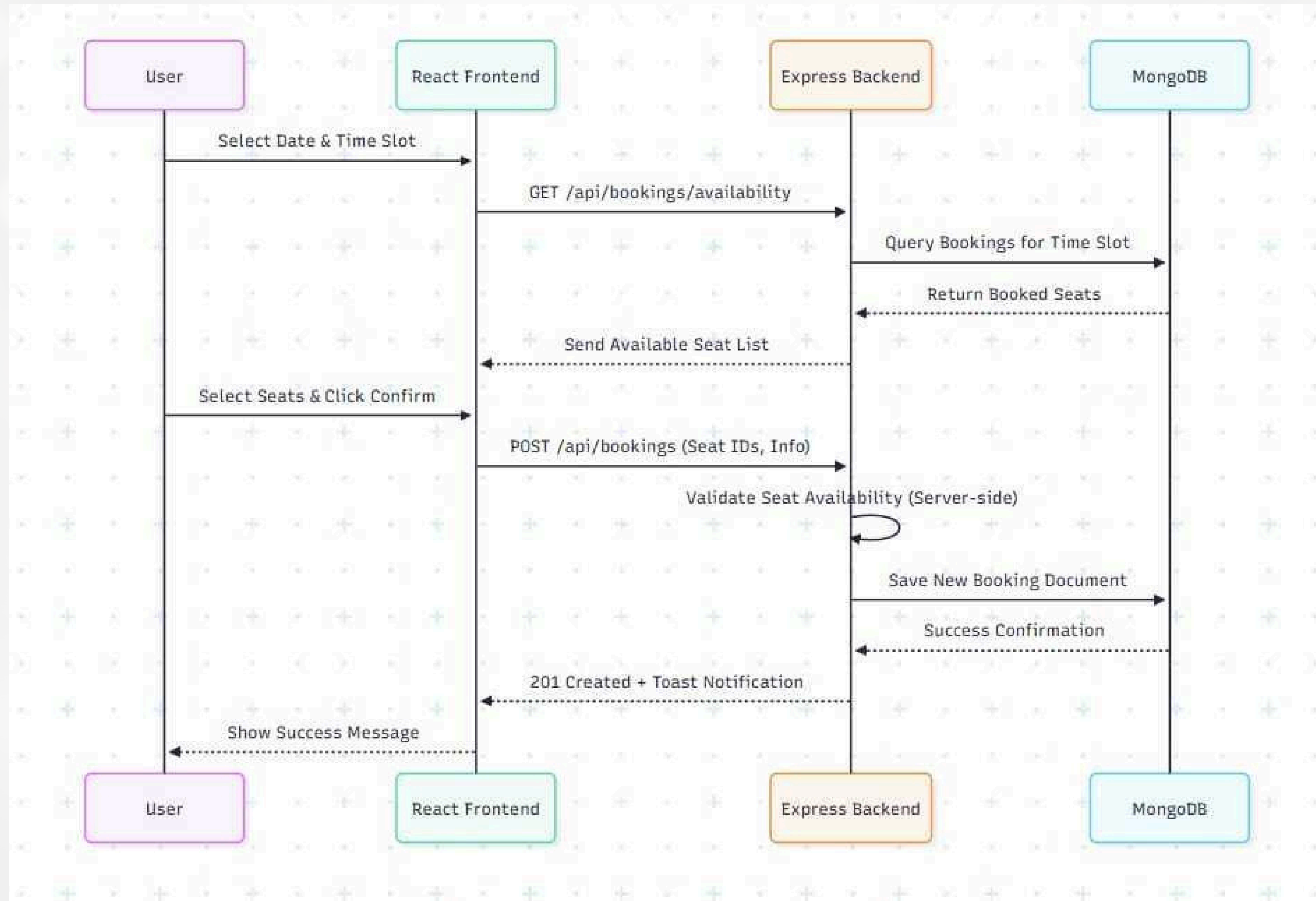
2

3

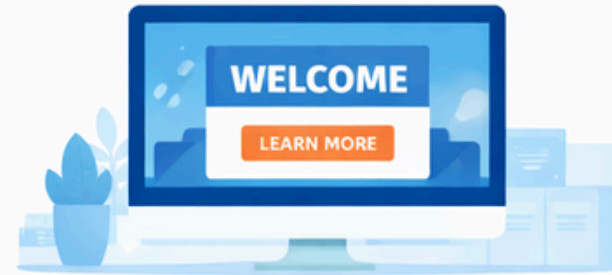
4

5

WORKFLOW ARCHITECTURE



Generic Landing Page



Initial generic page led directly to the main page



Manager Login



Added manager login with credentials stored in DB



IBM SSO Login



Implemented IBM SSO login for secure single sign-on



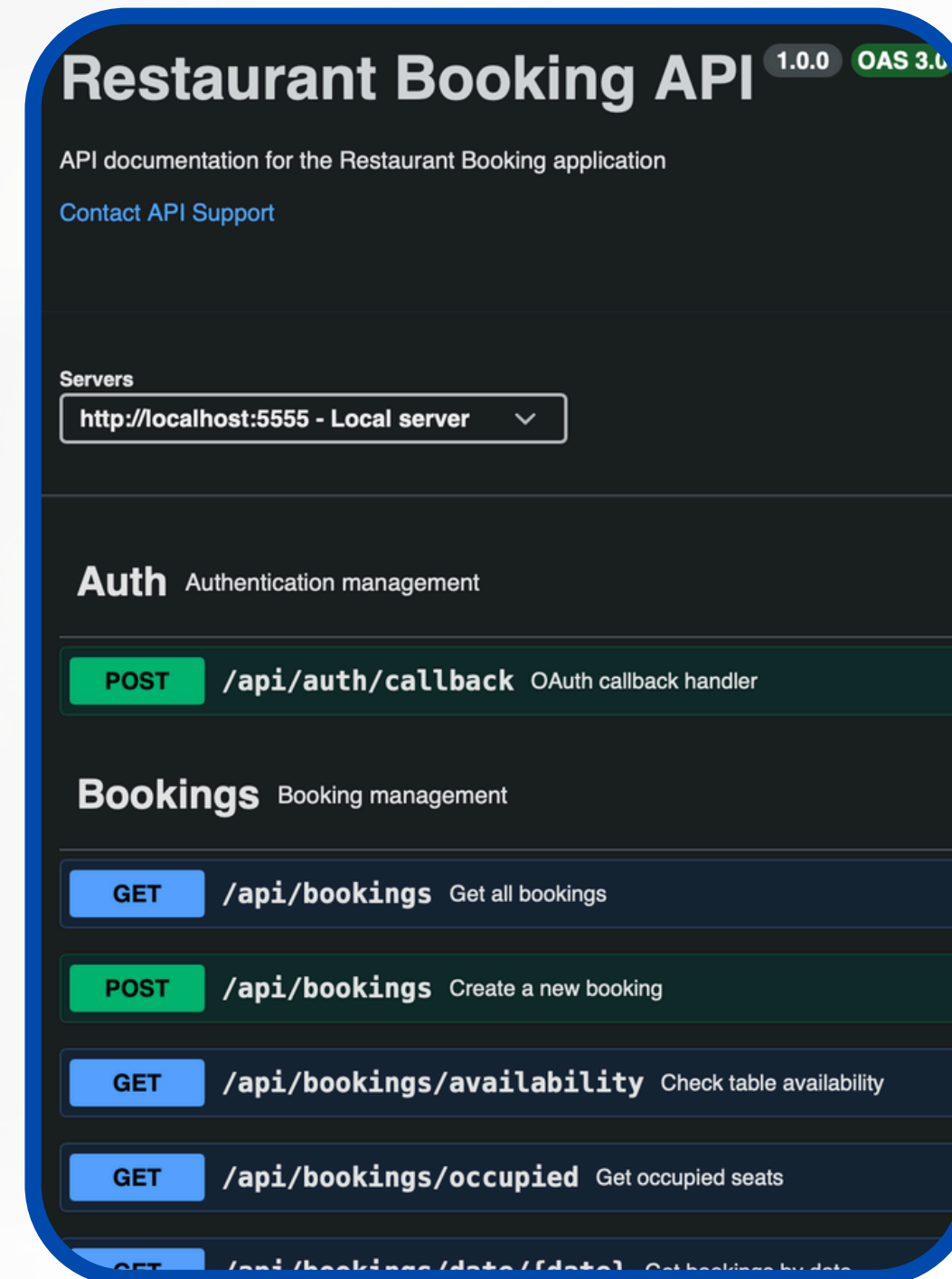
Demo User Login



AGILE SOFTWARE DEVELOPMENT

- Assigned responsibilities to each person and took continuous feedback of progress
- Features developed in small increments, we utilized iterative prototyping which was our topic for the presentation
- Regular integration of features to reduce merge conflicts
- Many many brainstorming sessions to decide which feature is realistic to implement or not
- To-do lists and progress feedbacks



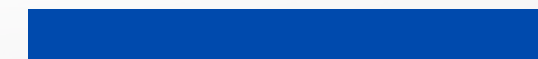


GIT AND GITHUB

PR merging from forked repos and feature based branching which ensured a clean history and surprisingly we had very less to none merge conflicts

SWAGGER

- Exposed backend APIs via Swagger UI
- Enables third-party developers to integrate easily
- Easy API exploration



SECURITY: DEV-SEC-OPS

IBM SSO AUTH

- Implemented IBM SSO authentication
- Manager credentials stored in database
- Passwords hashed before storage

SAST TOOLS

- npm audit**
npm audit fix
- Identified and fixed vulnerable dependencies

DAST TOOLS

- OWASP ZAP**
- Scans running application
 - Simulates real attacks
 - Tests exposed APIs and endpoints

Standard Mode

Sites

Quick StartRequestResponseRequester

Header: TextBody: Text

Contexts

Default Context

Sites

HTTP/1.1 404 Not Found

X-Powered-By: Express

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: *

Access-Control-Allow-Headers: *

Content-Security-Policy: default-src 'none'

X-Content-Type-Options: nosniff

Content-Type: text/html; charset=utf-8

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="utf-8">

<title>Error</title>

</head>

<body>

<pre>Cannot GET /robots.txt</pre>

HistorySearchAlertsOutputSpiderAJAX SpiderActive Scan

Alerts (9)

CSP: Failure to Define Directive with No Fallback (2)

Content Security Policy (CSP) Header Not Set

Cross-Domain Misconfiguration (4)

Missing Anti-clickjacking Header

Server Leaks Information via "X-Powered-By" HTTP Re

Timestamp Disclosure - Unix

X-Content-Type-Options Header Missing (2)

Information Disclosure - Suspicious Comments

Modern Web Application

Evidence:

CWE ID: 693

WASC ID: 15

Source: Passive (10055 - CSP)

Alert Reference: 10055-13

Input Vector:

Description:

The Content Security Policy fails to define one of the directives that has no fallback. Missing/excluding them is the same as allowin

Other Info:

The directive(s): frame-ancestors, form-action is/are among the directives that do not fallback to default-src.

Solution:

Ensure that your web server, application server, load balancer, etc. is properly configured to set the Content-Security-Policy header.

Reference:

https://www.w3.org/TR/CSP/

https://caniuse.com/#search=content+security+policy

https://content-security-policy.com/

Alert Tags:

Key	Value
OWASP_2021_A05	https://owasp.org/Top10/A05_2021-Security_I
POLICY_QA_STD	
POLICY_PENTEST	
SYSTEMIC	https://www.zaproxy.org/docs/desktop/addon

Alerts 0 4 3 2 Main Proxy: localhost:8080

Current Status 0 0 0

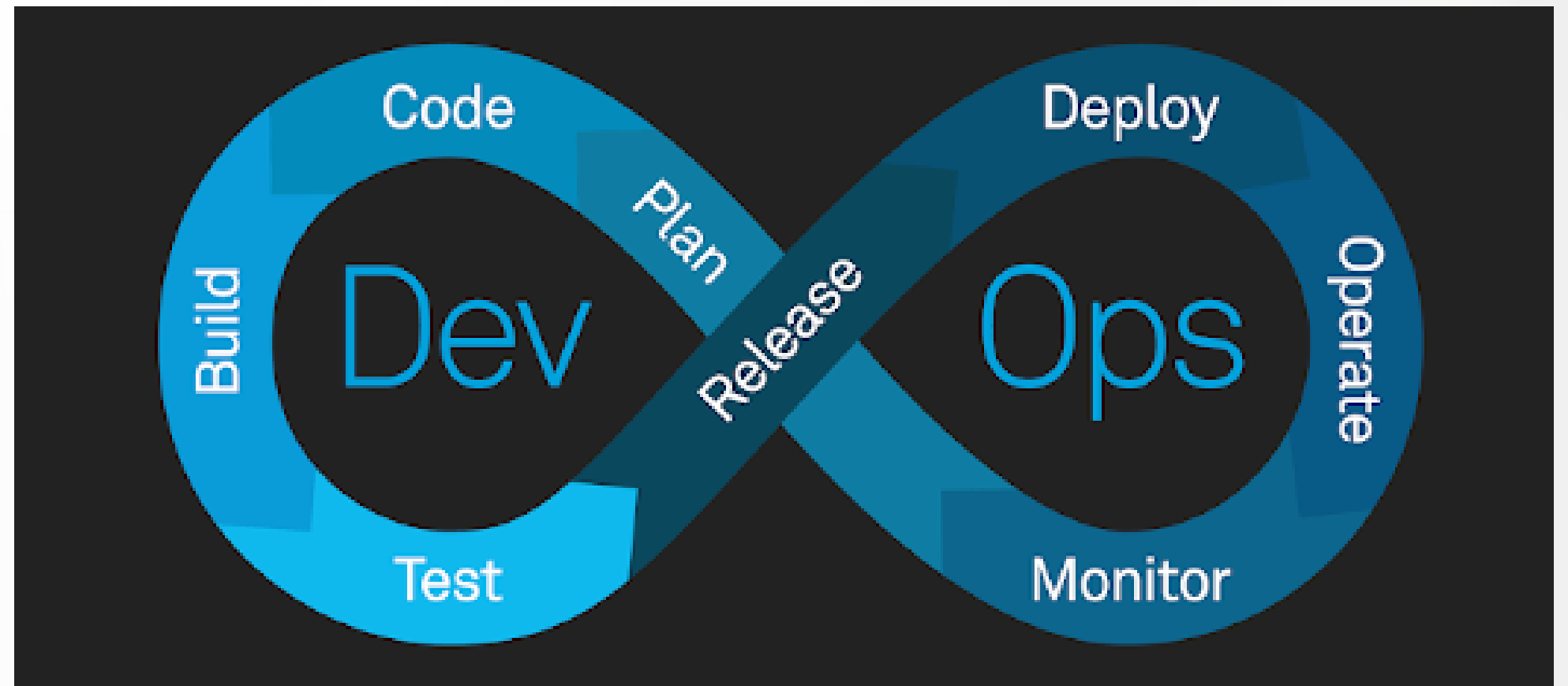
OWASP ZAP REPORT

The DAST scan reported only low-risk configuration and informational warnings related to security headers, with no critical vulnerabilities detected

- ✗ No SQL injection
- ✗ No auth bypass
- ✗ No data leak
- ✗ No critical issues

CI/CD PIPELINE

- **Automated Testing Pipeline:** GitHub Actions executes Jest unit tests automatically on every push and pull request to main and develop branches
- **Security Scanning:** CodeQL runs weekly security scans and on main branch updates to identify vulnerabilities early in development
- **Quality Assurance Checks:** Automated npm security audits, package.json validation, and sensitive file detection run on all PRs
- **Pull Request Automation:** PR validation includes change tracking, title checks, and automated welcome messages for contributors



UNIT TESTING

Performed backend unit testing using Jest

Covered key scenarios:

- Successful booking creation (201)
- Validation failure for missing inputs (400)
- Seat availability failure (400)
- Mocked database models (Booking, Restaurant) and external services (SMS, Email)
- Used mock request and response objects for isolated testing
- All unit tests executed successfully using npm test

```
PASS tests/booking.test.js
```

```
Booking Controller - Unit Test
```

- ```
✓ should create a booking and return 201 (6 ms)
✓ should return 400 when required fields are missing (1 ms)
✓ should return 400 if seats are unavailable (1 ms)
```

```
Test Suites: 1 passed, 1 total
```

```
Tests: 3 passed, 3 total
```

```
Snapshots: 0 total
```

```
Time: 2.15 s
```

```
Ran all test suites.
```

# OPENSIFT

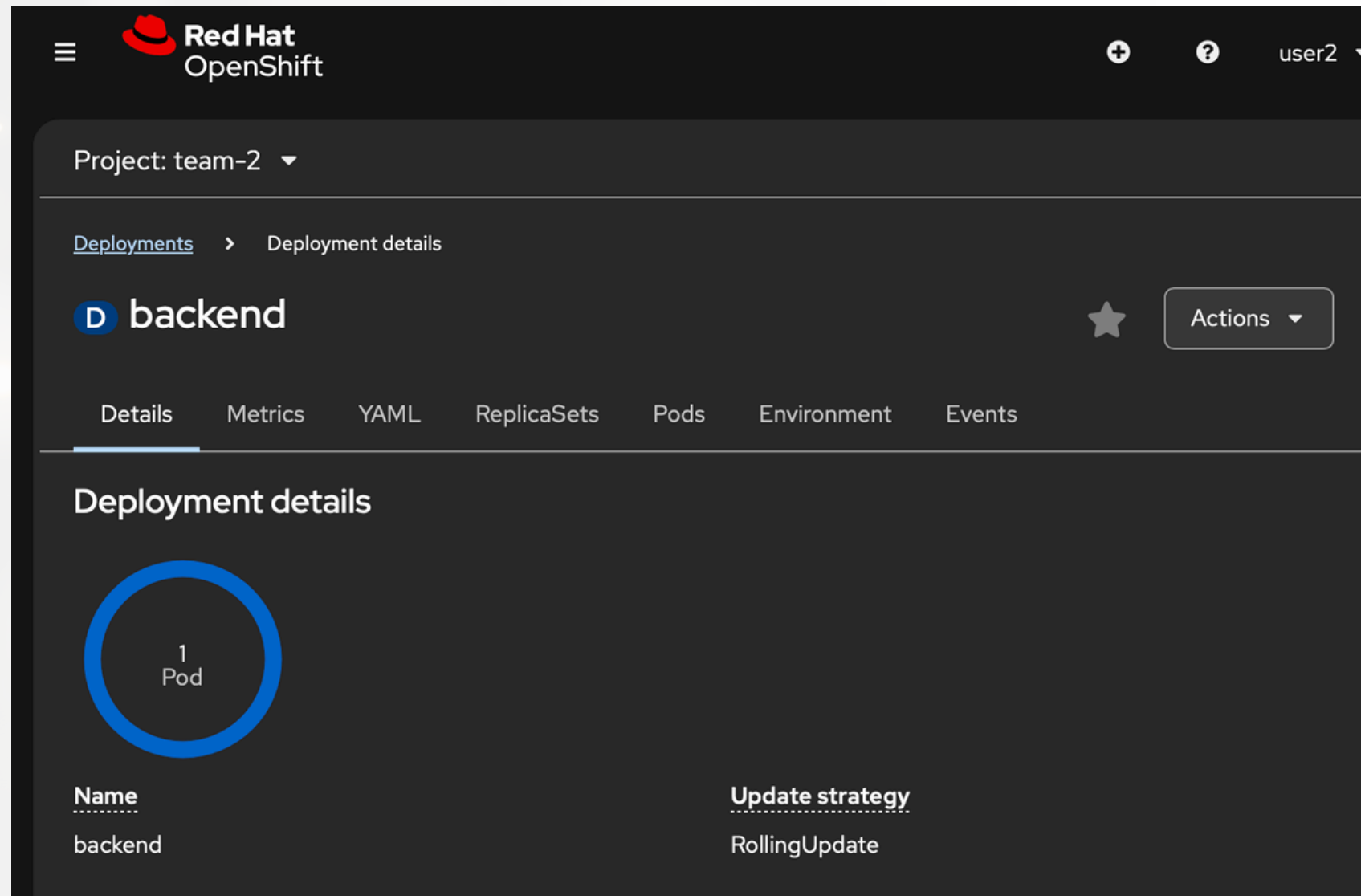
## DEPLOYMENT AND CONTAINERIZATION

### ***Containerization & Build***

- Frontend (React + Nginx) and Backend (Node.js + Express) were containerized using **Podman** and pushed to the OpenShift Image Registry

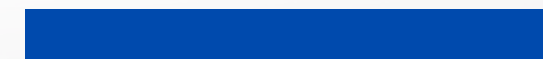
### ***OpenShift Deployment & Exposure***

- Containers were deployed using OpenShift Deployments and Services, and exposed externally through secure HTTPS Routes with environment-based configuration.



# TOOLS AND TECHNOLOGIES USED

- MERN for developement
- Git / GitHub for version control
- twilio / nodemailer for notifications
- IBM SSO for user auth
- jest for tesing
- Podman for containerization
- Openshift for deployment
- GitHub Actions
- SAST: npm audit
- DAST: OWASP ZAP
- Postman



# FUTURE SCOPE

---

1

Add analytics and reporting dashboards to provide insights into booking trends, peak hours, and seat utilization.

2

Add robust session management to control user sessions, token expiry, and logout across devices.

3

Enhance authentication and authorization using API keys with support for key rotation and rate limiting.

4

Study the position of the sun and the architecture of Rivera to see the seats which are having sunny and shadows in real time

5

Implement the waitlisting feature, where if a booking is cancelled then the next user in the queue gets that booking

# FUTURE SCOPE

---

6

Multiple restaurants, if this app is collaborated with hunger box.

7

Multiple cafeterias as well for eg, Rivera, pyramid, pine valley cafeteria etc.

8

To fix OWASP ZAP reported errors



**THANK YOU**

**VISIT US**



[www.reallygreatsite.com](http://www.reallygreatsite.com)